

JAVA PROGRAMMING

Comprehensive Study Material

Last Updated: June 30, 2026

TABLE OF CONTENTS

1. Object-Oriented Programming (OOP) Concepts
2. Core Language Features
3. Collections Framework
4. Exception Handling
5. Multithreading and Concurrency
6. I/O and File Handling
7. Generics and Type Safety
8. Functional Programming (Java 8+)
9. Memory Management and Garbage Collection
10. Database Connectivity (JDBC)

1. OBJECT-ORIENTED PROGRAMMING (OOP) CONCEPTS

Classes and Objects: Classes are blueprints for creating objects. Objects are instances of classes that encapsulate data (attributes) and behavior (methods). A class defines the structure and behavior that all objects of that class will have.

Inheritance: Inheritance allows a class (subclass) to inherit properties and methods from another class (superclass). It promotes code reuse and establishes an 'is-a' relationship. Java supports single inheritance for classes but multiple inheritance for interfaces.

Polymorphism: Polymorphism means 'many forms.' It allows objects to take multiple forms. Method overloading (compile-time polymorphism) and method overriding (runtime polymorphism) are the two types of polymorphism in Java.

Encapsulation: Encapsulation is the bundling of data and methods that operate on that data within a single unit (class). It hides internal implementation details and provides controlled access through getters and setters.

Abstraction: Abstraction hides complex implementation details and shows only the essential features. Abstract classes and interfaces are used to achieve abstraction in Java.

2. CORE LANGUAGE FEATURES

Variables and Data Types: Java has primitive data types (int, double, boolean, char, etc.) and reference data types (objects, arrays, strings). Variables store data values and must be declared with a type.

Operators: Java supports arithmetic (+, -, *, /), comparison (==, !=, <, >), logical (&&, ||, !), and bitwise operators (&, |, ^, ~). Operators combine operands to produce new values.

Control Flow: Conditional statements (if-else, switch) control program flow based on conditions. Loops (for, while, do-while) repeat code blocks. Break and continue statements modify loop behavior.

Methods: Methods are functions within a class that perform specific tasks. They have a return type, name, parameters, and body. Methods promote code reusability and modular programming.

3. COLLECTIONS FRAMEWORK

Lists: ArrayList and LinkedList implement the List interface. ArrayList provides fast random access but slower insertion/deletion. LinkedList provides fast insertion/deletion but slower random access.

Sets: HashSet, TreeSet, and LinkedHashSet implement the Set interface. Sets store unique elements with no duplicate values. HashSet is unordered, TreeSet is sorted, LinkedHashSet maintains insertion order.

Maps: HashMap, TreeMap, and LinkedHashMap implement the Map interface. Maps store key-value pairs. HashMap is unordered, TreeMap is sorted by keys, LinkedHashMap maintains insertion order.

Queues and Stacks: Queue (FIFO) and Stack (LIFO) are specialized collections. PriorityQueue, ArrayDeque, and LinkedList can be used as queues or stacks based on access patterns.

4. EXCEPTION HANDLING

Try-Catch-Finally: Try block contains code that might throw exceptions. Catch blocks handle specific exceptions. Finally block executes regardless of whether an exception occurred, used for cleanup.

Exception Types: Checked exceptions (IOException, SQLException) must be caught or declared. Unchecked exceptions (NullPointerException, ArrayIndexOutOfBoundsException) inherit from RuntimeException.

Custom Exceptions: You can create custom exceptions by extending Exception or RuntimeException. Custom exceptions provide specific error information relevant to your application.

5. MULTITHREADING AND CONCURRENCY

Threads: Threads are lightweight processes that run concurrently. A Java program can have multiple threads executing simultaneously. Threads are created by extending Thread class or implementing Runnable interface.

Synchronization: Synchronization prevents multiple threads from accessing shared resources simultaneously. Synchronized methods and blocks ensure thread safety. Monitors control access to shared data.

Concurrency Utilities: Java provides ExecutorService, CountdownLatch, Semaphore, and other concurrency utilities for managing multiple threads effectively.

6. I/O AND FILE HANDLING

Java provides streams for input/output operations. `FileInputStream/FileOutputStream` handle binary data. `FileReader/FileWriter` handle text data. `BufferedReader` provides efficient line-by-line reading.

7. GENERICS AND TYPE SAFETY

Generics enable type-safe collections and methods. Generic classes/methods accept type parameters. Bounded types restrict type parameters. Wildcards provide flexibility in generic programming.

8. FUNCTIONAL PROGRAMMING (JAVA 8+)

Lambda expressions provide concise syntax for functional interfaces. Streams API enables functional-style operations on collections. Method references provide shorthand for calling existing methods.

9. MEMORY MANAGEMENT

Java uses automatic garbage collection to free unused memory. Heap stores objects, stack stores primitive values and references. Understanding memory management prevents memory leaks and improves performance.

10. DATABASE CONNECTIVITY (JDBC)

JDBC (Java Database Connectivity) allows Java applications to interact with databases. DriverManager loads drivers, Connection establishes database connections, Statement executes SQL queries, ResultSet retrieves query results.